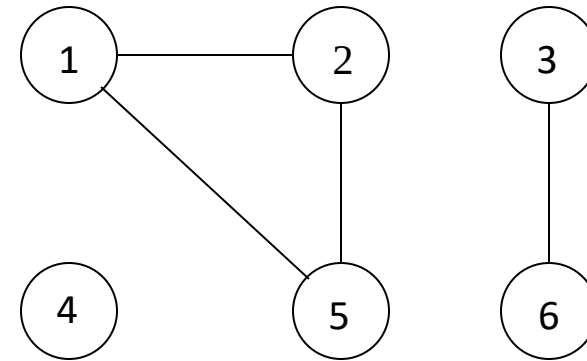
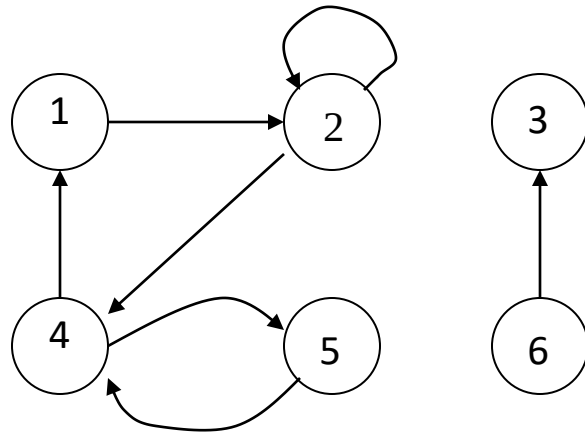


# GRAPHS

# What is a Graph?

- Informally a *graph* is a set of nodes joined by a set of lines or arrows.



# Definitions

Graph is a set of Vertices and Edges

$G := (V, E)$

- Where  $V$  is a set of vertices
- $E$  is a set of edges

Edge is a tuple  $(V_i, V_j)$  such that  $V_i, V_j \in V$

# Definitions

Graph is a set of Vertices and Edges

$G := (V, E)$

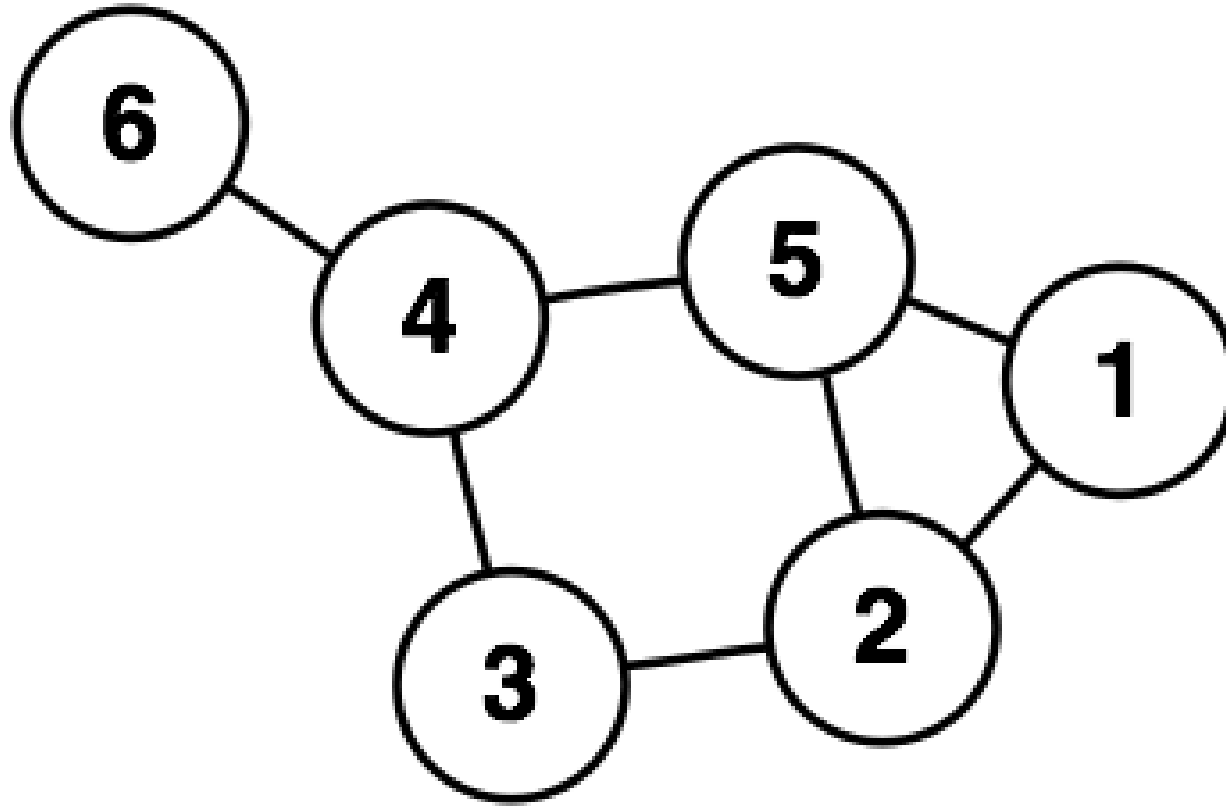
- Vertex

- Basic Element
- Drawn as a *node* or a *dot*.
- **Vertex set** of  $G$  is usually denoted by  $V(G)$ , or  $V$

- Edge

- A set of two elements
- Drawn as a line connecting two vertices, called end vertices, or endpoints.
- The edge set of  $G$  is usually denoted by  $E(G)$ , or  $E$ .

# Example



- $V := \{1, 2, 3, 4, 5, 6\}$
- $E := \{\{1, 2\}, \{1, 5\}, \{2, 3\}, \{2, 5\}, \{3, 4\}, \{4, 5\}, \{4, 6\}\}$

# Applications of Graphs

- Maps Eg. Road map
- Computer Networks
- Electrical Circuits

# Types of Graphs

- Undirected / Directed (diagraph)
- Weighted Edge / Non-weighted Edge
- Undirected Weighted
- Undirected Non-weighted
- Directed Weighted
- Directed Non-weighted

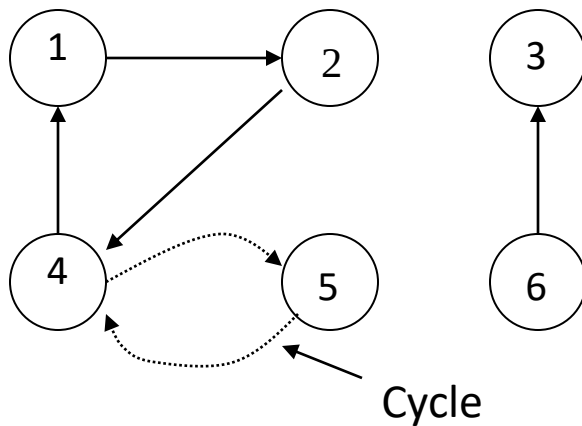
# Simple Graphs

*Simple graphs* are graphs without multiple edges or self-loops.



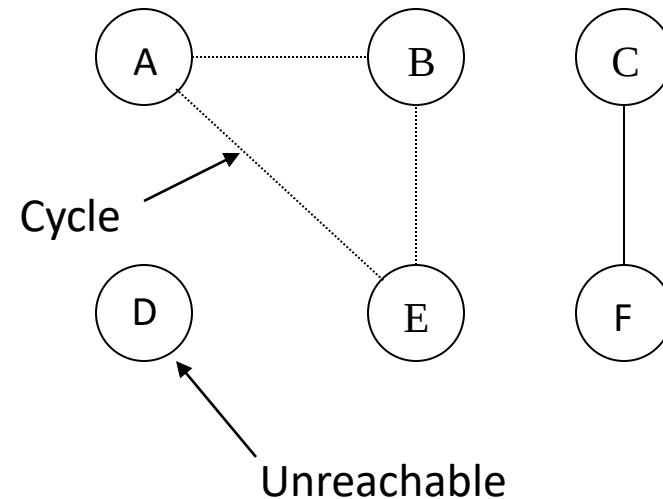
# Path

- A *path* is a sequence of vertices such that there is an edge from each vertex to its successor.
- A path is ***simple*** if each vertex is distinct.



**Simple path from 1 to 5**  
**= [ 1, 2, 4, 5 ]**

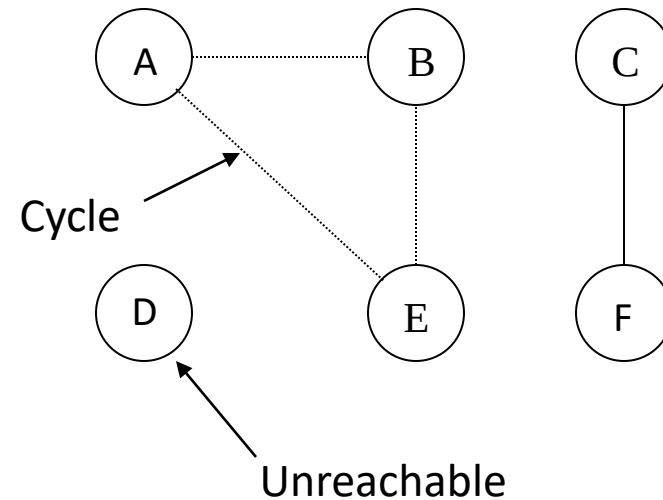
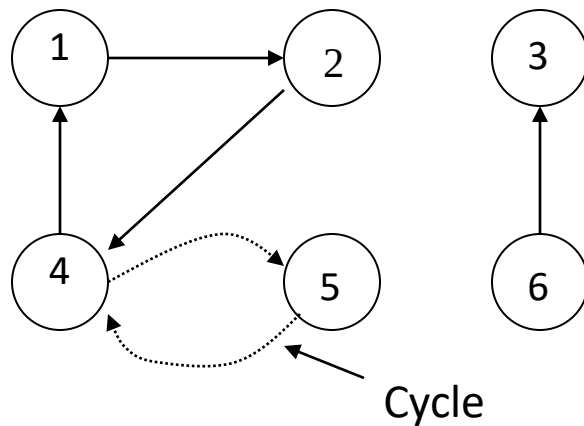
Our text's alternates the vertices and edges.



If there is path  $p$  from  $u$  to  $v$  then we say  $v$  is **reachable** from  $u$  via  $p$ .

# Cycle

- A path from a vertex to itself is called a ***cycle***.
- A graph is called ***cyclic*** if it contains a cycle;
  - otherwise it is called ***acyclic***

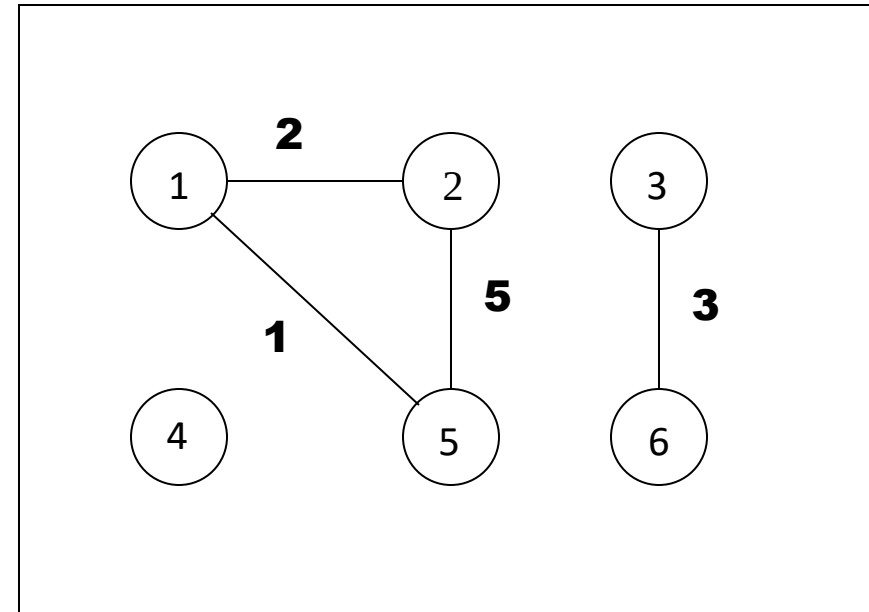
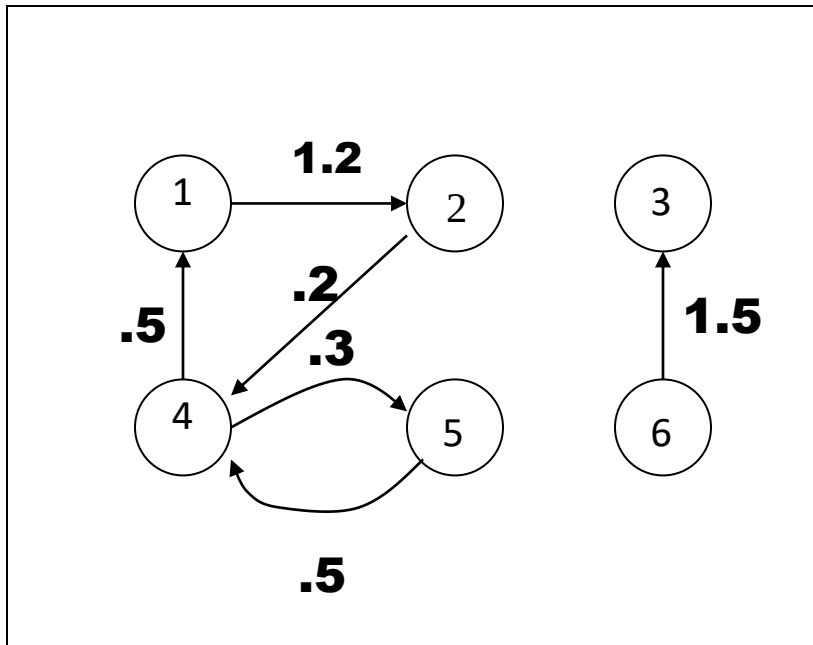


# Connectivity

- is ***connected*** if
  - you can get from any node to any other by following a sequence of edges OR
  - any two nodes are connected by a path.
- A directed graph is ***strongly connected*** if there is a directed path from any node to any other node.

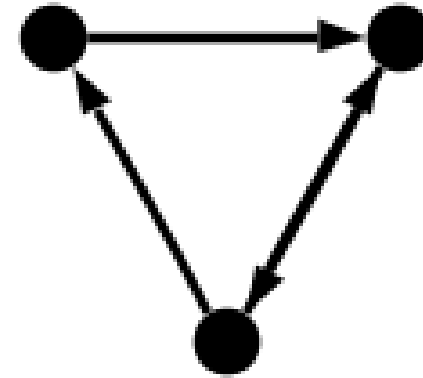
# A *weighted graph*

- is a graph for which each edge has an associated ***weight***, usually given by a ***weight function***  $w: E \rightarrow \mathbf{R}$ .



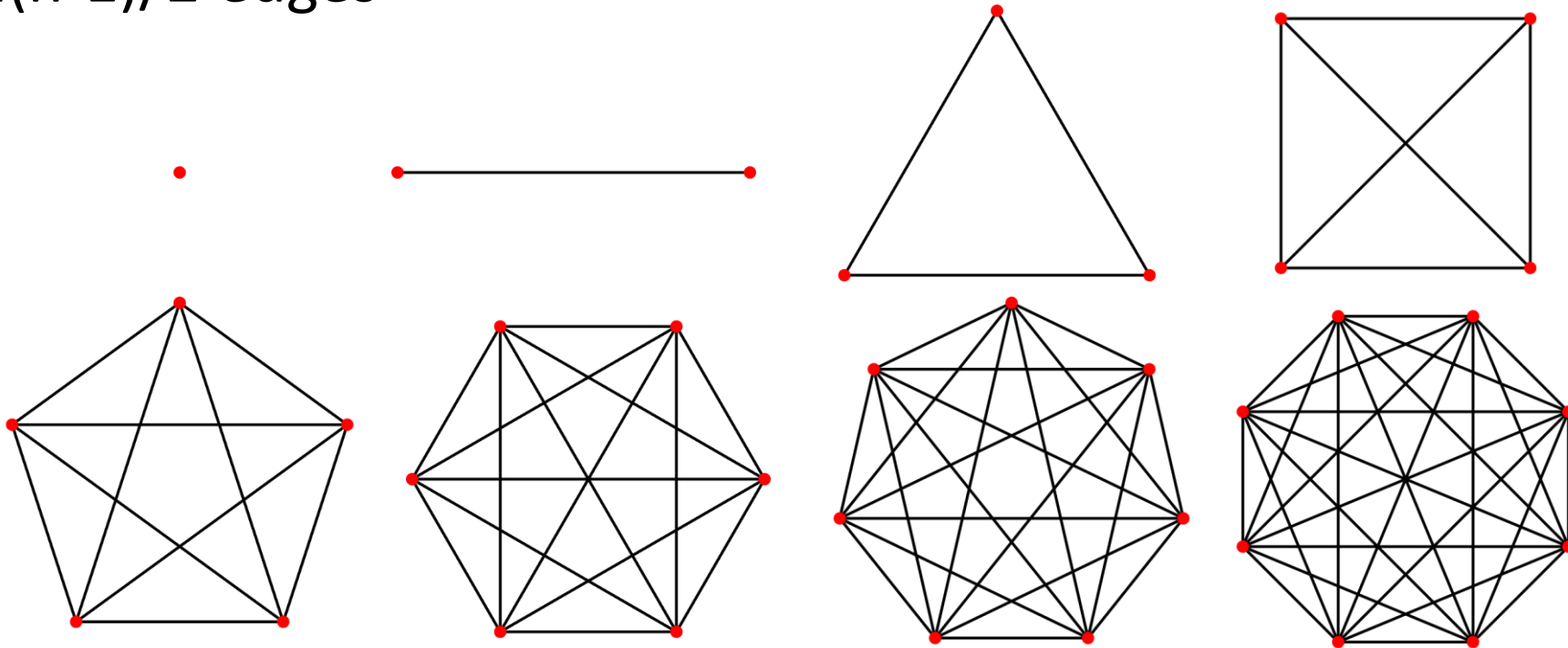
# Directed Graph (digraph)

- Edges have directions
  - An edge is an *ordered* pair of nodes



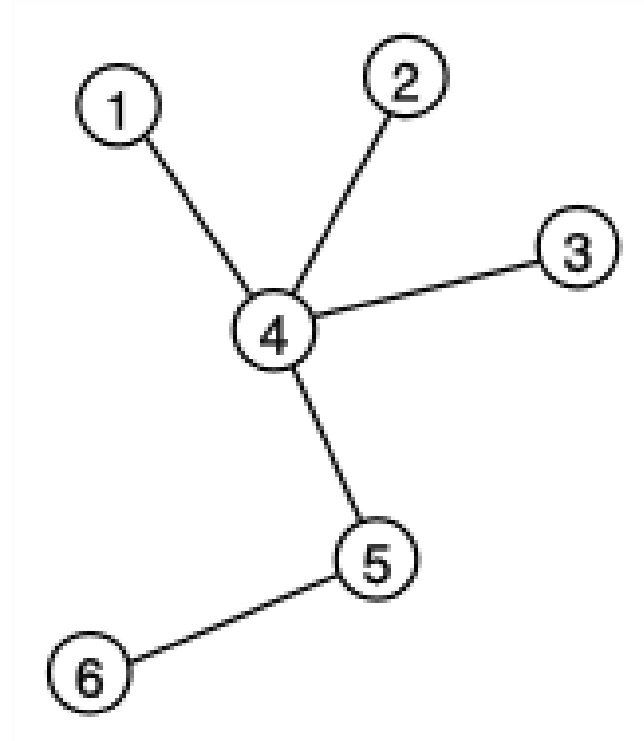
# Complete Graph

- Denoted  $K_n$
- Every pair of vertices are adjacent
- Has  $n(n-1)/2$  edges

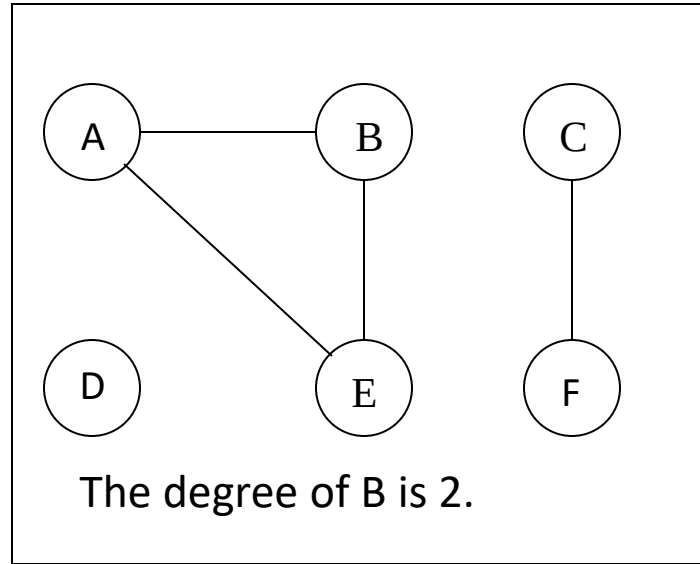


# Tree

- Connected Acyclic Graph
- Two nodes have *exactly* one path between them



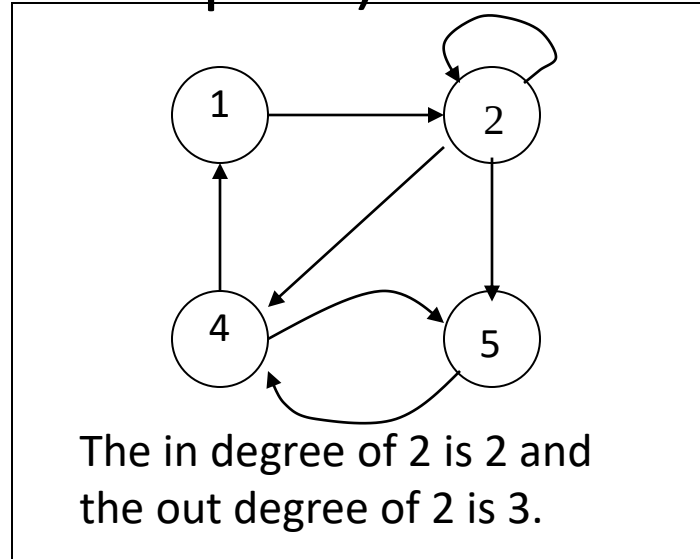
# Degree



- Number of edges incident on a node



# Degree (Directed Graphs)



- In degree: Number of edges entering
- Out degree: Number of edges leaving
- Degree = indegree + outdegree

# Degree: Simple Facts

- If  $G$  is a digraph with  $m$  edges, then

$$\sum \text{indeg}(v) = \sum \text{outdeg}(v) = m = |E|$$

- If  $G$  is a graph with  $m$  edges, then

$$\sum \text{deg}(v) = 2m = 2 |E|$$

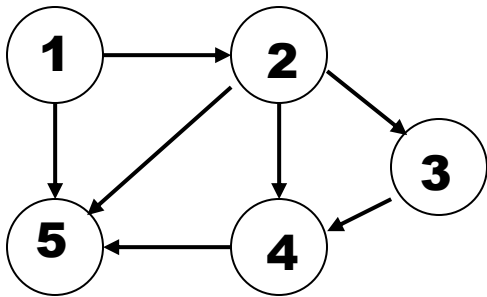
- Number of Odd degree Nodes is even

# Representation (Matrix)

## Adjacency Matrix

- $V \times V$  matrix
- Boolean values (adjacent or not) Or Edge Weights

# Adjacency Matrix Representation for a Directed Graph

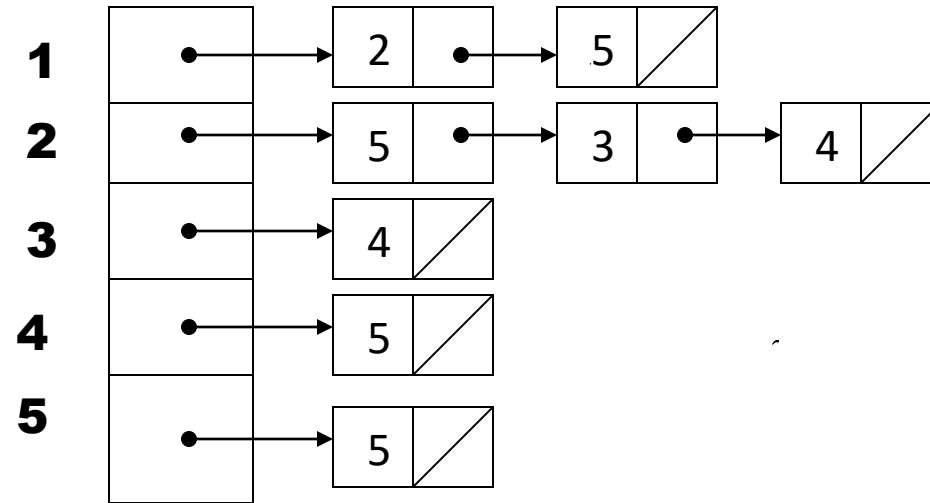
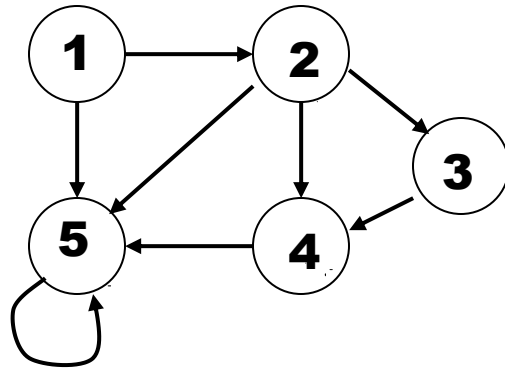


|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 0 | 1 |
| 2 | 0 | 0 | 1 | 1 | 1 |
| 3 | 0 | 0 | 0 | 1 | 0 |
| 4 | 0 | 0 | 0 | 0 | 1 |
| 5 | 0 | 0 | 0 | 0 | 0 |

- ***Adjacency-list representation***

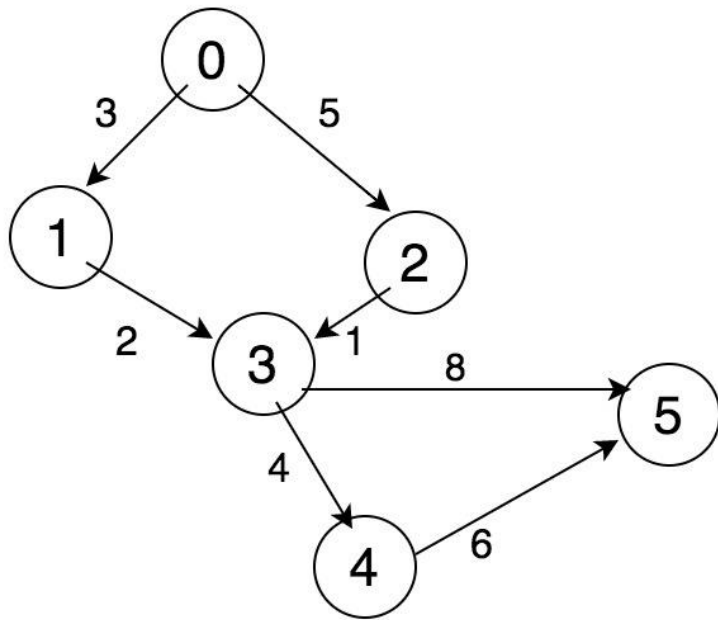
- an array of  $|V|$  lists, one for each vertex in  $V$ .
- For each  $u \in V$ ,  $ADJ[u]$  points to all its adjacent vertices.

# Adjacency-list representation for a directed graph.

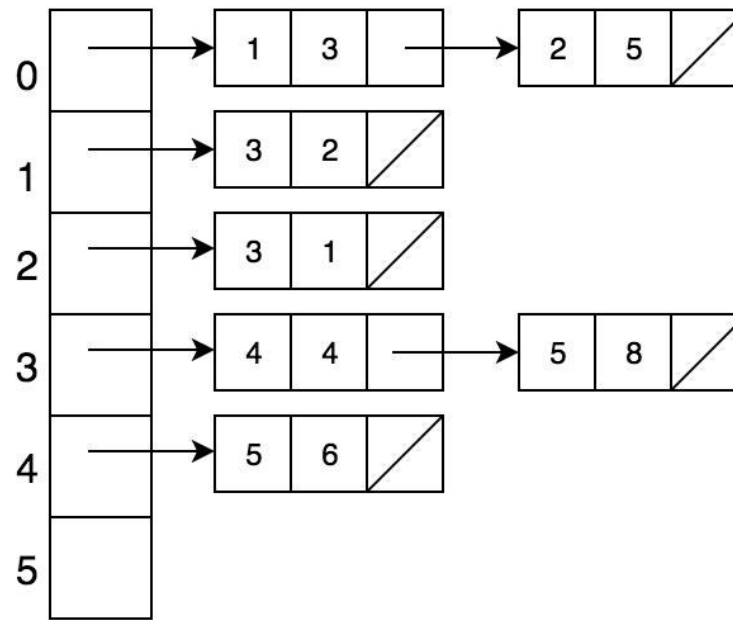


# Adjacency-list representation for a Weighted directed graph.

**Directed Graph**

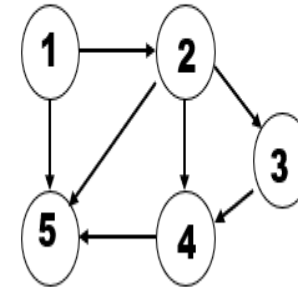


**Adjacency List Representation**



# Graph Implementation

```
void initializeGraph(Graph *graph, int numVertices) {  
    graph->numVertices = numVertices;  
    int i, j;  
    for (i = 0; i < numVertices; i++) {  
        for (j = 0; j < numVertices; j++) {  
            graph->vertices[i][j] = 0;  
        }  
    }  
}
```



|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 0 | 1 |
| 2 | 0 | 0 | 1 | 1 | 1 |
| 3 | 0 | 0 | 0 | 1 | 0 |
| 4 | 0 | 0 | 0 | 0 | 1 |
| 5 | 0 | 0 | 0 | 0 | 0 |



# Graph Implementation

```
void addEdge(Graph *graph, int src, int dest, int weight) {  
    graph->vertices[src][dest] = weight;  
    graph->vertices[dest][src] = weight;  
}
```

# Graph Implementation

```
void displayGraph(Graph *graph) {  
    int i, j;  
    for (i = 0; i < graph->numVertices; i++) {  
        for (j = 0; j < graph->numVertices; j++) {  
            printf("%d ", graph->vertices[i][j]);  
        }  
        printf("\n");  
    }  
}
```

# Graph Implementation

```
int isAdjacent(Graph *graph, int vertex1, int vertex2) {  
    if (graph->vertices[vertex1][vertex2] != 0) {  
        return 1;  
    } else {  
        return 0;  
    }  
}
```

# Graph Implementation

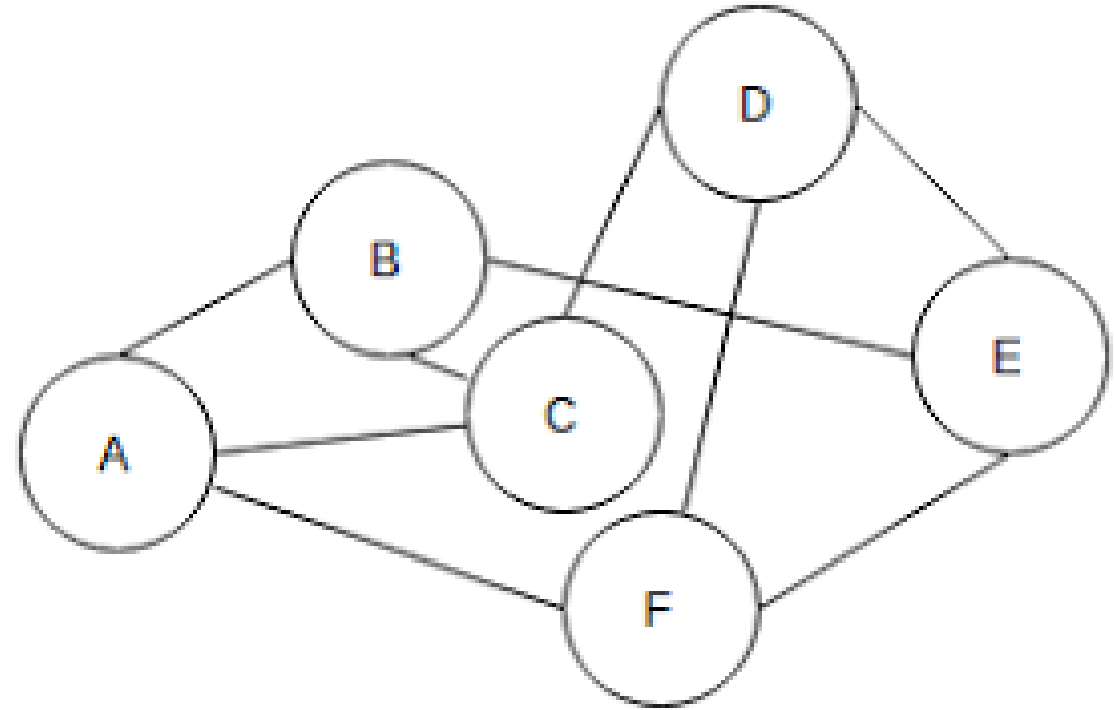
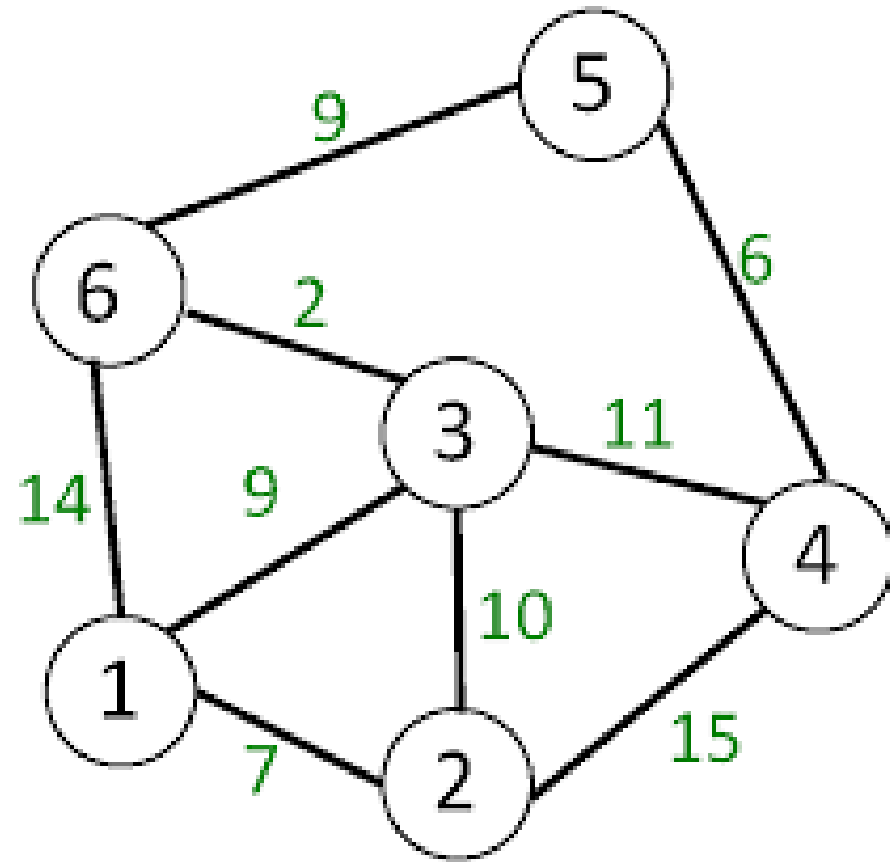
```
int getDegree(Graph *graph, int vertex) {  
    int degree = 0;  
    int i;  
    for (i = 0; i < graph->numVertices; i++) {  
        if (graph->vertices[vertex][i] != 0) {  
            degree++;  
        }  
    }  
    return degree;  
}
```

# GRAPH TRAVERSAL

# Graph traversals/search

- Graph traversal means visiting every vertex and edge exactly once in a well-defined order. While using certain graph algorithms, you must ensure that each vertex of the graph is visited exactly once. The order in which the vertices are visited are important and may depend upon the algorithm or question that you are solving.
- During a traversal, it is important that you track which vertices have been visited. The most common way of tracking vertices is to mark them.

How to Traverse vertices of these graphs :

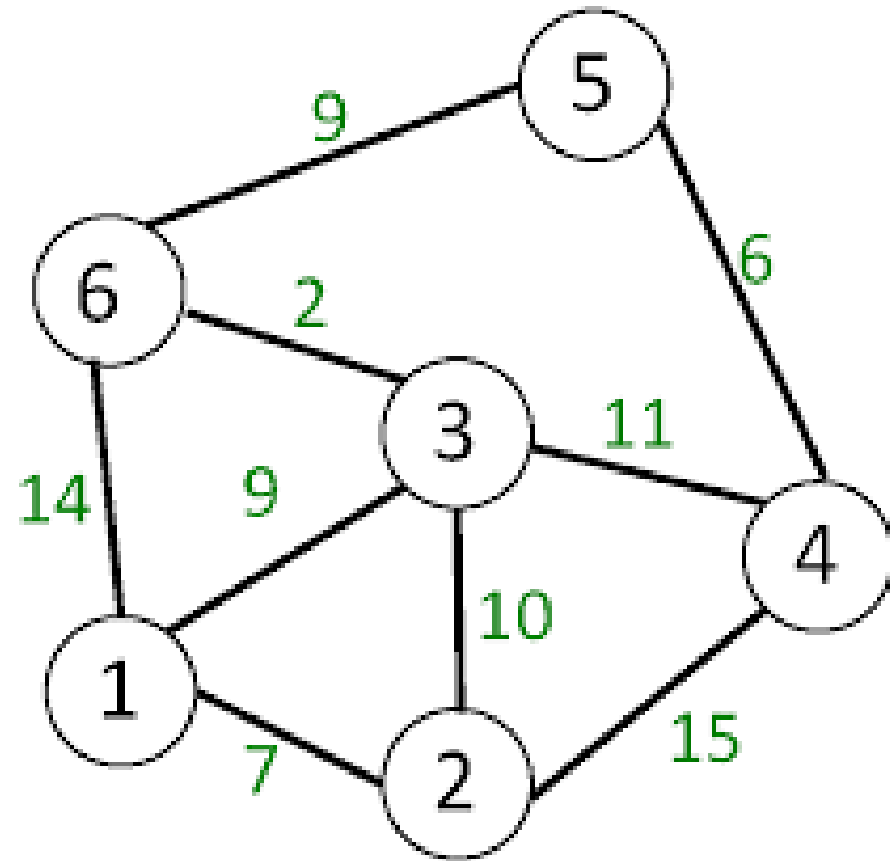


# Breadth First Search

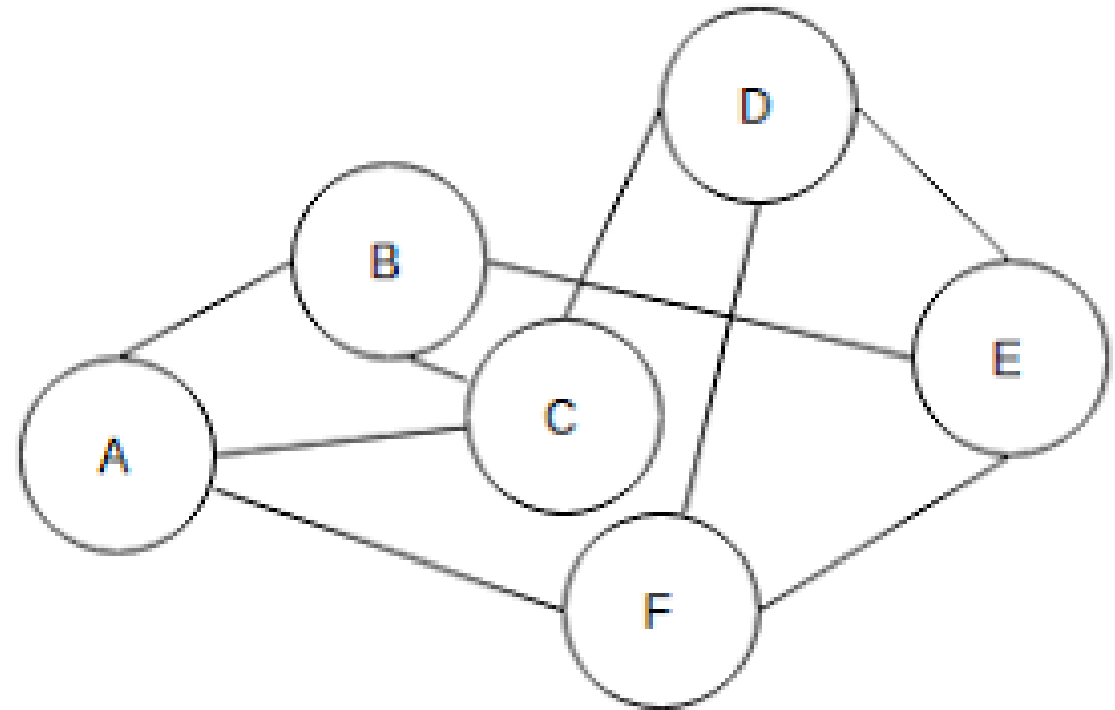
- BFS is a traversing algorithm where you should start traversing from a selected node (source or starting node) and traverse the graph level wise thus exploring the neighbour nodes (nodes which are directly connected to source node). You must then move towards the next-level neighbour nodes.
- As the name BFS suggests, you are required to traverse the graph breadthwise as follows:
  1. First move horizontally and visit all the nodes of the current layer
  2. Move to the next layer



BFS Traversal of these graphs :



1, 6, 3, 2, 5, 4



A,B, C, F, E, D

# BFS -ALGO

BFS (G, s)                   //Where G is the graph and s is the source node

let Q be queue.

Q.enqueue( s ) //Inserting s in queue until all its neighbour vertices are marked.

mark s as visited.

while ( Q is not empty)

    //Removing that vertex from queue, whose neighbour will be visited now

    v = Q.dequeue( )

    //processing all the neighbours of v

    for all neighbours w of v in Graph G

        if w is not visited

            Q.enqueue( w )                   //Stores w in Q to further visit its neighbour

            mark w as visited.

# Depth First Search

- The DFS algorithm is a recursive algorithm that uses the idea of backtracking. It involves exhaustive searches of all the nodes by going ahead, if possible, else by backtracking.
- Here, the word backtrack means that when you are moving forward and there are no more nodes along the current path, you move backwards on the same path to find nodes to traverse. All the nodes will be visited on the current path till all the unvisited nodes have been traversed after which the next path will be selected.

This recursive nature of DFS can be implemented using stacks. The basic idea is as follows:

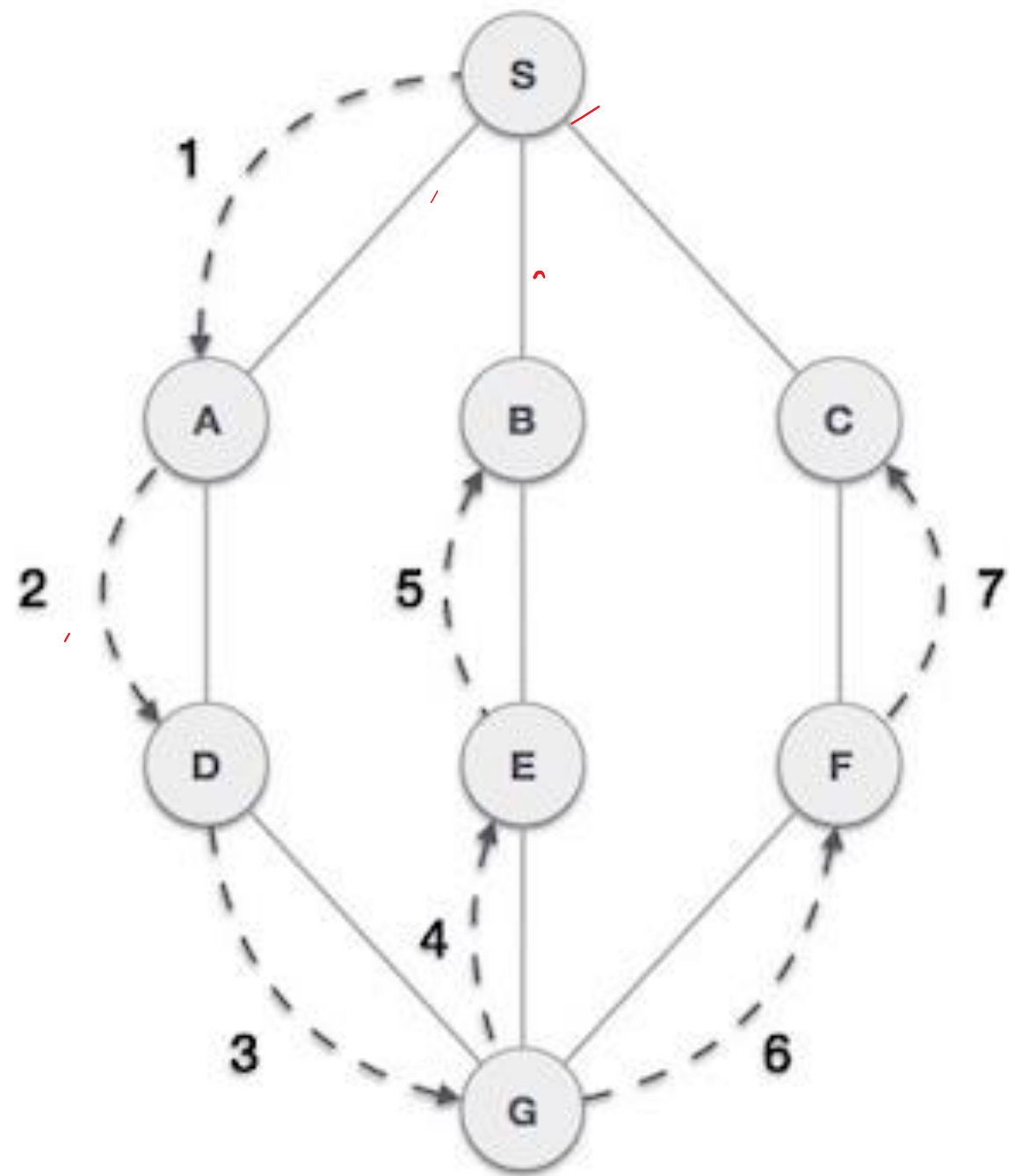
Pick a starting node and push all its adjacent nodes into a stack.

Pop a node from stack to select the next node to visit and push all its adjacent nodes into a stack.

Repeat this process until the stack is empty. However, ensure that the nodes that are visited are marked. This will prevent you from visiting the same node more than once. If you do not mark the nodes that are visited and you visit the same node more than once, you may end up in an infinite loop.

# DFS

- **Rule 1** – Visit the adjacent unvisited vertex. Mark it as visited. Display it. Push it in a stack.
- **Rule 2** – If no adjacent vertex is found, pop up a vertex from the stack. (It will pop up all the vertices from the stack, which do not have adjacent vertices.)
- **Rule 3** – Repeat Rule 1 and Rule 2 until the stack is empty.



DFS-iterative (G, s): //Where G is graph and s is source vertex

let S be stack

S.push( s ) //Inserting s in stack

mark s as visited.

while ( S is not empty):

    //Pop a vertex from stack to visit next

    v = pop(S )

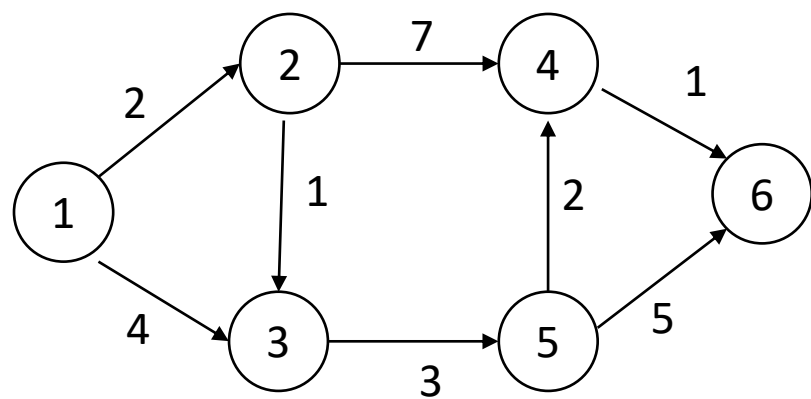
    //Push all the neighbours of v in stack that are not visited

    for all neighbours w of v in Graph G:

        if w is not visited :

            S.push( w )

        mark w as visited





# DFS

```
void dfs(Graph *graph, int vertex, int *visited) {  
    visited[vertex] = 1;  
    int i;  
    for (i = 0; i < graph->numVertices; i++) {  
        if (graph->vertices[vertex][i] != 0 && !visited[i]) {  
            dfs(graph, i, visited);  
        }  
    }  
}
```

# BFS

---

```
void bfs(Graph *graph, int vertex, int *visited)
{
    int i=0;
    Queue q;
    init(&q);
    enqueue(&q, vertex);
    visited[vertex]=1;
    while (q->rear)
    {
        for (i=0; i<graph->numVertices; i++)
        {
            if (graph->vertices[vertex][i] != 0 && visited[i]==0)
            {
                enqueue(&q, i);
                visited[i]=1;
            }
        }
        printf("%d\t", q->front->data);
        dequeue(&q);
        if (q->front)
            vertex=q->front->data;
    }
    return;
}
```

# Is Connected

```
int isConnected(Graph *graph) {  
    int visited[MAX_VERTICES];  
    int i;  
    for (i = 0; i < graph->numVertices; i++) {  
        visited[i] = 0;  
    }  
  
    dfs(graph, 0, visited);  
  
    for (i = 0; i < graph->numVertices; i++) {  
        if (!visited[i]) {  
            return 0;  
        }  
    }  
  
    return 1;  
}
```